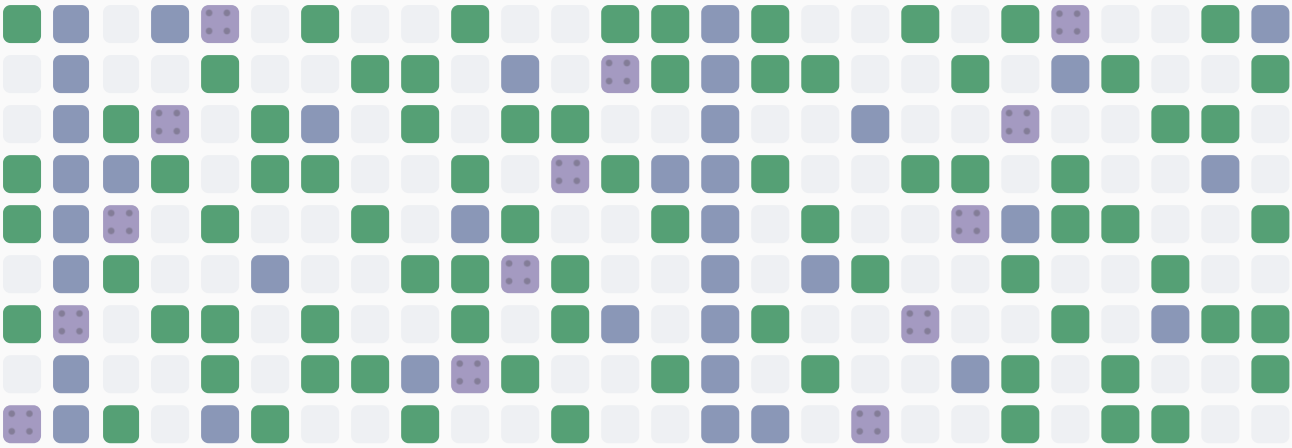


CONTAINERDAYS & AI CONTEXT SINGAPORE · 28 OCT 2026 · HANDOUT

The Memory-Bandwidth Wall

Experiments you can run yourself

bandwidth ÷ bytes per token = tok/s

A practical reference for tuning vLLM: estimate the limit, measure your workload, choose an experiment and decide whether to keep the change.

Start with one model on one GPU. Change one setting at a time, compare throughput with latency, and check task quality before deploying. The examples show commands and what to read back; adapt the model, lengths and load to your own traffic.

WORK IN THIS ORDER

01 Know your ceiling

02 Measure MBU

03 Find the knee under load

04 Apply the levers in one experiment

05 Gate quality

06 Dashboards and alerts

07 The fleet: llm-d

08 Eval recipe and change ticket

Marcel Wysocki · marcel.w.wysocki@gmail.com · linkedin.com/in/marcel-w-wysocki

Speaking for myself, not for Red Hat.

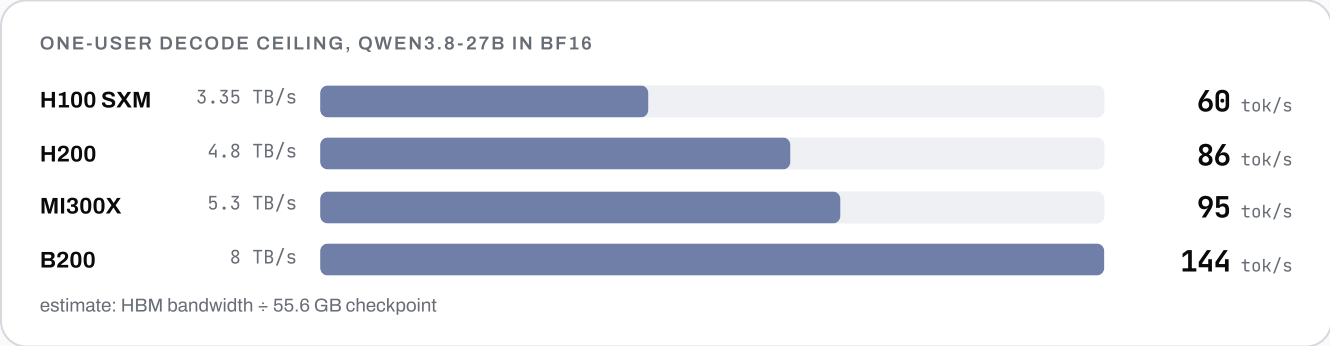
01 Know your ceiling

Estimate bytes read per token from the checkpoint: for a dense model, roughly the weights used during decode; for an MoE model, the routed part at batch 1 (and close to all experts under diverse load). This is a checkpoint-based ceiling estimate, not a measured memory trace; embeddings, vision and auxiliary tensors can overstate per-step reads.

```bash
# Small index only; no weights downloaded. Requires curl and jq.
MODEL=Qwen/Qwen3.8-27B
REV=1d4bf0f2ff6012fd82039f2fa52739d0dd7c60c0
curl -fsSL "https://huggingface.co/$MODEL/resolve/"\
"$REV/model.safetensors.index.json" |
  jq '.metadata.total_size / 1e9'
# 55.562855904 GB of tensors in the whole checkpoint
```

The index reports **55.56 GB of tensors**, excluding file headers. Use this as a first estimate. For text decode, exclude unused vision and prediction modules, and count one input-embedding row rather than the whole lookup table. The output projection still counts. For this checkpoint, that refinement gives **51.25 GB per decode step**. Use the same byte definition for the ceiling and MBU.

Ceiling (tok/s, one user) = HBM bandwidth in bytes/s ÷ bytes per token. HBM bandwidth comes from the spec sheet: H100 SXM 3.35 TB/s, H200 4.8, MI300X 5.3, B200 8.



02 Measure MBU

Run the examples in Bash, in the Python environment where vLLM is installed. The snippets use Python 3, curl and jq. Keep the server in one terminal and the load generator in another; stop the previous test server before changing its configuration. The custom-data replay also needs pandas in that environment: `uv pip install pandas`.

The general vLLM examples use the 0.31.0 CLI; features needing a different release are labeled. Check your installed version with `vllm --version` and pin it for each comparison.

```bash
vllm serve Qwen/Qwen3.8-27B --port 8000 --enforce-eager \
  --no-enable-prefix-caching --no-enable-chunked-prefill \
  --max-model-len 8192 --max-num-batched-tokens 8192

# Quick diagnostic; download, compilation and a full repeated test take longer
vllm bench serve --backend vllm --model Qwen/Qwen3.8-27B \
  --dataset-name random --random-input-len 128 --random-output-len 128 \
  --num-prompts 4 --num-warmups 2 --max-concurrency 1 \
  --ignore-eos --temperature 0 --seed 1234 \
  --percentile-metrics ttft,tpot,itl --metric-percentiles 50,95
```

Use one user, short context and speculative decoding off. MBU = (1000 ÷ **Mean TPOT** in ms) ÷ the matching weight-read ceiling estimate. Use TPOT, not `Output token throughput`, which includes the wait for the first token. Keep the seed fixed to repeat the same requests; the server command disables prefix caching.

- Low MBU: kernels, launch overhead, batch shape. Tuning can help.
- High MBU: the bus may be the limit. There is no universal threshold. Confirm in a profile before you stop tuning overhead. Fewer bytes, more tokens per read, or a faster bus still raise tok/s.

Worked MBU example: a 51.25 GB text-decode weight read on a 273 GB/s GPU gives a ceiling of **5.33 tok/s**. A Mean TPOT of **222.5 ms** gives **4.49 decode tok/s**, or about **84% estimated weight MBU**. This estimate assumes one read of each weight per step. KV, recurrent state, cache residency and kernel rereads can change actual traffic; MBU is not a hardware-counter reading.

Warm the server before timing. Model loading, compilation and kernel tuning can take much longer than a short diagnostic. Use the same warm-up for baseline and candidate, and repeat longer tests before drawing conclusions about sustained load.

To repeat your own one-user diagnostic, leave the prefix-disabled server from the start of this section running. In a second terminal, save three runs and summarize their JSON output with Python's standard library:

```bash
MBU_DIR="results/mbu-$(date +%Y%m%d-%H%M%S)"
for n in 1 2 3; do
  vllm bench serve --backend vllm --model Qwen/Qwen3.8-27B \
    --dataset-name random --random-input-len 128 --random-output-len 128 \
    --num-prompts 16 --num-warmups 1 --max-concurrency 1 \
    --ignore-eos --temperature 0 --seed 1234 \
    --percentile-metrics ttft,tpot,itl --metric-percentiles 50,95 \
    --save-result --save-detailed --result-dir "$MBU_DIR" \
    --result-filename "run$n.json"
done
```

Summarize the saved runs in the same terminal:

```bash
python3 - "$MBU_DIR" <<'PYCODE'
import json, math, sys
from pathlib import Path
from statistics import median
runs = [json.loads(p.read_text()) for p in Path(sys.argv[1]).glob("run*.json")]
assert len(runs) == 3, "Need three completed runs"
for r in runs:
    assert r["completed"] == 16 and r["failed"] == 0
    assert not any(r["errors"])
for key in ("mean_tpot_ms", "p95_ttft_ms", "output_throughput"):
    values = [r[key] for r in runs]
    assert all(math.isfinite(v) and v > 0 for v in values)
    print(f"{key}: median {median(values):.2f}; range {min(values):.2f}–{max(values):.2f}")
PYCODE
```

Use median Mean TPOT in the MBU formula above. The range shows repeatability across these three batches; it is not a confidence interval. This test measures serving behavior, not coding success or a production SLO.

Save a baseline you can compare later

Use a separate terminal for the server. Stop that test server before starting a different configuration on the same GPU and port. Record the GPU, model revision, engine version and startup KV capacity. Keep the same context limit, warm-up and arrival pattern in each comparison.

```bash
vllm --version
nvidia-smi --query-gpu=name,memory.total,driver_version --format=csv

vllm bench serve --backend vllm --model Qwen/Qwen3.8-27B \
  --dataset-name random --random-input-len 1024 --random-output-len 512 \
  --num-prompts 80 --num-warmups 2 --max-concurrency 1 --seed 1234 \
  --percentile-metrics ttft,tpot,itl --metric-percentiles 50,95,99 \
  --save-result --save-detailed --result-dir results/baseline-run1
```

Run the candidate with the same seed and lengths into `results/candidate-run1`, then repeat both with the chosen cache policy. Save each run in its own result directory. Detailed results can include model outputs; keep them with the task data. Four short prompts are a quick MBU check; 80 longer prompts provide a more sustained comparison, but neither establishes a tail-latency SLO. Use a longer load test for that.

Low MBU: profile before you tune

vLLM drives the profiler itself with `--profiler-config` (profiling guide). Profile the same server configuration and workload that produced the low MBU. This example preserves the diagnostic settings above and adds Nsight Systems:

```bash
VLLM_WORKER_MULTIPROC_METHOD=spawn nsys profile \
  --trace-fork-before-exec=true --cuda-graph-trace=node \
  --capture-range=cudaProfilerApi --capture-range-end repeat \
  vllm serve Qwen/Qwen3.8-27B --port 8000 --enforce-eager \
  --no-enable-prefix-caching --no-enable-chunked-prefill \
  --max-model-len 8192 --max-num-batched-tokens 8192 \
  --profiler-config.profiler cuda

# Benchmark terminal: same diagnostic workload, now with capture enabled
vllm bench serve --backend vllm --model Qwen/Qwen3.8-27B \
  --dataset-name random --random-input-len 128 --random-output-len 128 \
  --num-prompts 4 --num-warmups 2 --max-concurrency 1 \
  --ignore-eos --temperature 0 --seed 1234 \
  --percentile-metrics ttft,tpot,itl --metric-percentiles 50,95 --profile

# Stop the profiled server after the benchmark to write the report
nsys stats report1.nsys-rep
```

Check the time spent in decode kernels and the gaps between them (launch overhead). The PyTorch profiler is the same switch: `--profiler-config '{"profiler": "torch", "torch_profiler_dir": "./vllm_profile"}'`, then `curl -X POST localhost:8000/start_profile` and `/stop_profile` around your requests. The profile endpoints are for development only (security guidance): never enable `--profiler-config` on a production server.

03 Find the knee under load

GuideLLM, a vLLM subproject, runs load sweeps and produces latency and throughput reports. Use it against your running server to find where extra traffic stops buying useful throughput.

```bash
uv pip install guidellm==0.8.1
guidellm run --backend kind=openai_http,target=http://localhost:8000 \
  --profile kind=sweep,sweep_size=10,strategy_type=poisson \
  --constraint kind=max_duration,seconds=60 \
  --data kind=synthetic_text,prompt_tokens=1024,output_tokens=512 \
  --output kind=json --output kind=html
```

At each rate, read TTFT p95 and per-request TPOT p95, and check where output tok/s stops rising. Keep ITL for stream smoothness.

Separate capacity from request rate

A fixed-concurrency ladder holds a number of users in flight; a fixed request rate keeps arrivals independent of how slowly replies finish. Use both before choosing an admission limit. For this Bash ladder, run the server below with prefix caching disabled so repeated seeds cannot create prefix hits between steps. The rest of the serving setup stays fixed.

```bash
# Server terminal: isolate load from prefix reuse
vllm serve Qwen/Qwen3.8-27B --max-model-len 32768 \
  --no-enable-prefix-caching
```

```bash
# Benchmark terminal
for BENCH_USERS in 1 4 16 64; do
  vllm bench serve --backend vllm --model Qwen/Qwen3.8-27B \
    --dataset-name random --random-input-len 1024 --random-output-len 512 \
    --num-prompts 256 --num-warmups 2 --max-concurrency "$BENCH_USERS" \
    --seed 1234 --percentile-metrics ttft,tpot,itl --metric-percentiles 50,95 \
    --save-result --result-dir "results/concurrency-$BENCH_USERS"
done

# Open-loop arrivals: 8 requests/s, without a client concurrency cap
vllm bench serve --backend vllm --model Qwen/Qwen3.8-27B \
  --dataset-name random --random-input-len 1024 --random-output-len 512 \
  --num-prompts 256 --num-warmups 2 --request-rate 8 --seed 1234 \
  --percentile-metrics ttft,tpot,itl --metric-percentiles 50,95 \
  --save-result --result-dir results/rate-8
```

Record output tok/s, P95 TTFT, P95 per-request TPOT, failed requests and preemptions. The knee is where throughput flattens or the latency target fails. A client concurrency cap can hide server overload by holding arrivals at the client. Random prompts test serving mechanics; they do not score model quality.

Read throughput and latency together

More concurrency can increase total output while making every request slower. Stop increasing load when throughput flattens, the latency target fails, or errors and preemptions grow. A high token rate is useful only when requests still finish in time.

Keep three comparisons separate:

- **Concurrency:** change how many requests can be in flight, keeping the serving configuration fixed.
- **Arrival rate:** change how often requests arrive. Include idle time in throughput and check whether the client actually achieved the requested rate.
- **Cache capacity:** keep the weight checkpoint and load fixed. Compare equal cache bytes when testing formats; compare token capacity separately. Smaller KV can fit more tokens without making the kernels faster.

Use run medians and ranges to see repeatability. A handful of short batches cannot establish a production tail-latency target. Run longer at realistic traffic levels, including bursts and a representative mix of context lengths.

04 Apply the levers in one experiment

Choose a workload that matches the service you want to improve. Short prompts help diagnose decode overhead; recorded conversations reveal the costs of long histories, prefix reuse and concurrent requests. Keep the workload fixed within each comparison.

Work through the following steps, skipping mechanisms that do not address the measured bottleneck:

1. **Establish the baseline.** bf16, one client, compilation/graphs/chunking/prefix caching disabled. Estimate the ceiling and measure MBU.

2. **Restore normal serving features.** Enable compilation, CUDA graphs, chunked prefill and prefix caching in separate steps. Record first-token wait as well as decode speed.

3. **Shrink the weight read.** Switch to a supported quantized checkpoint; keep the other settings and prompts fixed.

4. **Test smaller KV.** Hold weights fixed. Compare speed at short and long context, plus reported startup capacity. Keep a negative result if those bytes were not the bottleneck.

5. **Compare speculation.** Test native MTP widths and compatible external drafters. Select on measured time per token and total throughput, not acceptance rate alone.

6. **Serve the coding team.** Give different tasks the same source prefix. Compare cache off, unprimed and warm batches, then sweep concurrency and token budget. Report total throughput beside each request's latency.

7. **Test reuse beyond GPU memory.** Compare CPU or disk reload with recomputing the prefix. Include transfer cost and memory use, and verify the returned answers.

8. **Apply the quality gate.** Keep the useful changes together, rerun the actual final stack, and score your own tasks. A performance run is not a quality verdict.

On one GPU, cache-aware routing has no placement choice. Use section 7 when scaling to multiple replicas.

Replay recorded coding requests

The public SWE-bench Pro trace corpus reports median input **63,917 tokens**, median output **246**, and mean output **520**. The important change from the short diagnostic is much longer input and repeated history; outputs do not all need to be longer. Trace corpus and statistics.

A serving replay sends saved requests and controls response lengths. It measures serving behavior; it does not execute tools or test whether an agent completes its task.

Choose lengths from your own request logs when you have them. Preserve the mixture of short tool calls, longer answers and growing conversation histories. Record how much of each input is actually uncached: two requests with 80,000 input tokens can impose very different prefill work. Keep the prepared requests and per-request output lengths fixed across configurations, and report first-token wait alongside decode latency and total throughput.

Prepare your own `traces.jsonl`: one JSON object per line, with `prompt` (a string already formatted with the target model's chat template) and `output_tokens` (a positive integer). Preserve tool messages and their order, and keep private inputs local. Start one of the server configurations below, then run:

```bash
# Set this to the model ID served by your running server.
MODEL=nvidia/Qwen3.8-27B-NVFP4
CACHE_SALT=$(python3 -c 'import secrets; print(secrets.token_hex(32))')
curl --fail http://localhost:8000/health
vllm bench serve --backend vllm --model "$MODEL" \
  --dataset-name custom --dataset-path traces.jsonl --skip-chat-template \
  --custom-output-len -1 \
  --num-prompts 0 --disable-shuffle --max-concurrency 8 \
  --num-warmups 0 --ready-check-timeout-sec 0 \
  --request-rate inf --temperature 0 --ignore-eos --seed 1234 \
  --extra-body "{\"cache_salt\":\"$CACHE_SALT\"}" \
  --percentile-metrics ttft,tpot,itl --metric-percentiles 50,95 \
  --save-result --save-detailed --result-dir results/my-replay-run1
```

`--num-prompts 0` uses all custom records. `--custom-output-len -1` preserves each record's `output_tokens` instead of overriding it with the CLI default. `--skip-chat-template` avoids formatting twice; `--ignore-eos` holds output lengths fixed for this serving test. The health check sends no generation request; the two zero-valued options prevent the client from priming a prefix before timing. Warm up kernels separately under a different salt. Use a new salt and result directory for each repeat, and retain all three runs. Record the cache state before each batch and check actual input/output lengths, errors, token hits and preemptions. The CLI saves throughput and latency measurements directly. vLLM benchmark CLI.

Define cache states explicitly. **Cache off** disables prefix caching on the server. **Unprimed** starts without cached history, but later requests in the same batch may reuse earlier work. **Warm** primes the shared histories before timing; include that preparation cost when your workload cannot amortize it.

For warm comparisons, use the same salt for priming and measurement within a batch, then a new salt for the next batch. Check actual prefix-token hits: limited capacity can evict primed histories. Keep later inputs fixed from the saved trace; generated responses do not become tool results for subsequent requests. Evaluate that behavior separately with live tasks.

Start with explicit serving settings

This example uses an NVFP4 checkpoint and native multi-token prediction (MTP) on compatible hardware. Choose context and memory limits that fit your GPU; they are experiment controls, not recommended defaults for every deployment.

```bash
vllm serve nvidia/Qwen3.8-27B-NVFP4 \
  --max-model-len 131072 --gpu-memory-utilization 0.70 \
  --max-num-batched-tokens 8192 --max-num-seqs 16 \
  --enable-prefix-caching --enable-chunked-prefill --kv-cache-dtype auto \
  --speculative-config '{"method":"mtp","num_speculative_tokens":1}'
```

First measure the same configuration without `--speculative-config`. Then add MTP and compare at your intended concurrency. Once individual changes look useful, test the complete configuration together.

Lever reference

LEVER	TRY	CHECK
Prefix reuse	On by default. A/B it: `--no-enable-prefix-caching`. Share within an approved team or group. Where isolation is required, enforce a secret, unpredictable `cache_salt` per trust group; different salts prevent prefix reuse across groups	`sum(rate(vllm:prefix_cache_hits_total[5m])) / sum(rate(vllm:prefix_cache_queries_total[5m]))`
Chunked prefill	On by default where supported. Scheduling-step token budget: `--max-num-batched-tokens 2048 | 8192 | 16384` (vLLM 0.32.0 serving defaults: 8192 on Spark/GB10 and H100/H200; 16384 on B200)	p95 `vllm:request_time_per_output_token_seconds` against p95 TTFT; `vllm:num_requests_waiting`
Weight bytes: NVFP4	`vllm serve nvidia/Qwen3.8-27B-NVFP4` (no flag; read from the checkpoint)	log `Model loading took X GiB`; Marlin indicates W4A16 fallback rather than native W4A4
Weight bytes: online FP8	`--quantization fp8_per_tensor` on a supported GPU	quality gate, section 5
KV bytes: FP8	`--kv-cache-dtype fp8`; Experiment C also shows how to keep sliding-window layers wide	log `GPU KV cache size: N tokens` before and after; warning `Using KV cache scaling factor 1.0` means no calibrated scales
KV bytes: NVFP4	vLLM 0.32.0: `--kv-cache-dtype nvfp4` with a compatible FlashInfer backend. Attention KV uses block scales; hybrid recurrent state stays wide	Startup token capacity, TPOT at your context length and task quality; keep the weight checkpoint fixed
KV bytes: unequal K/V bits	`--kv-cache-dtype turboquant_k8v4` (keys 8-bit, values 4-bit)	KV cache size, Mean TPOT and quality against FP8; lower bit count does not guarantee faster kernels
Speculative decode, no training	Prompt lookup through `--speculative-config`; complete example in Experiment D	log `SpecDecoding metrics: Mean acceptance length`; Mean TPOT (ITL hides the gain)
Speculative decode, model's own heads	`--speculative-config`; start with one native MTP token, as in the serving example above	same; try 1 before 4: more tokens per pass can mean less speed
Speculative decode, trained speculator	`--speculative-config` with a compatible EAGLE-3 or DSpark checkpoint; examples in Experiment D (documentation)	Acceptance length, draft cost, TPOT and total throughput on your saved prompts
Batch size	`--max-num-seqs 256` (default 1024 on H100 and up)	`vllm:num_preemptions_total`; KV usage near 1.0
Memory for the KV cache	`--gpu-memory-utilization` (default 0.92)	log `GPU KV cache size`
CPU KV offload	Native: `--kv-offloading-size 64` (GiB, summed across TP ranks). For LMCache disk reuse, configure capacity on its standalone server; see Experiment E	`sum(rate(vllm:external_prefix_cache_hits_total[5m])) / sum(rate(vllm:external_prefix_cache_queries_total[5m]))`
CPU KV offload a cache-aware router can see	Native backend: publish self-describing KV events over ZMQ (complete flags in Experiment E). Both settings default off; without them, the router counts offloaded prefixes as misses	the router's own cache-hit metric shows CPU-tier hits; vLLM v0.31.0 offload and KV-event configuration
Tensor parallel	`--tensor-parallel-size 2`	MBU per GPU; Mean TPOT vs TP=1

Budget memory before increasing the cache

Start with physical GPU memory, then allow for weights, runtime buffers, compilation, graph capture and temporary startup allocations. The remainder can hold KV. On unified-memory systems, CPU and GPU allocations draw from the same physical pool.

For example, an **85.2 GiB server budget**, **51.7 GiB of weights** and an assumed **4 GiB of runtime memory** leave about **29.4 GiB for KV**. A hybrid model with **64 KiB of attention KV per token plus 147 MiB of recurrent state per conversation** fits roughly **45 conversations at 8k context**, **3 at 128k**, or **7 at 128k with FP8 attention KV**. These are raw-payload estimates; block padding, extra state checkpoints and temporary buffers reduce usable capacity. Compare them with the startup log.

Keep a physical-memory reserve and observe startup as well as serving. A container limit may not contain every GPU allocation on a unified-memory system. A failed startup is a capacity result, not a throughput measurement. Reduce the relevant allocation or startup workload before retrying; changing several controls at once makes the comparison harder to interpret.

Experiment A: repeated prefixes, caching off versus on

Run one of these server commands at a time. Both get the same warm-up and the same synthetic shared prefix; the treatment is prefix caching. This is a warm-prefix experiment, not a cold first-request claim.

```bash
# Before: recompute the prefix
vllm serve Qwen/Qwen3.8-27B --dtype bfloat16 \
  --max-model-len 32768 --no-enable-prefix-caching

# After: allow prefix reuse (the normal default)
vllm serve Qwen/Qwen3.8-27B --dtype bfloat16 \
  --max-model-len 32768 --enable-prefix-caching
```

```bash
# Run once per configuration; use a different result directory for each
vllm bench serve --backend vllm --model Qwen/Qwen3.8-27B \
  --dataset-name random --random-prefix-len 4096 \
  --random-input-len 1024 --random-output-len 256 \
  --num-prompts 80 --num-warmups 2 --max-concurrency 4 --seed 1234 \
  --percentile-metrics ttft,tpot --metric-percentiles 50,95 \
  --save-result --result-dir results/prefix-off-run1
```

The total input here is **5,120 tokens**: 4,096 fixed prefix plus 1,024 random tail. Rename the candidate directory to `results/prefix-on-run1`. Compare TTFT and prefix **token** hit rate; a hit skips prefill, while decode still reads the context.

For a coding workload, put a fixed repository snapshot, harness and tool definitions first, then vary the task. Save fully formatted requests in `traces.jsonl` as described above. Compare cache off, unprimed and warm batches at the same concurrency, with the same output lengths and warm-up. Repeat at the load you expect to serve.

Report first-token wait, prefix-token hit rate and total throughput together. Include priming cost when the shared history will be used only once.

Choose who should share. Developers using the same harness, tool definitions and system prompt can reuse that exact prefix across requests. Put stable shared instructions first and changing task details last. Reuse needs matching token prefixes, not merely similar prompts. A shared random `cache_salt` lets an approved team or organization reuse prefixes while separate salts isolate other groups. A single trusted deployment can omit the salt. Where isolation is required, enforce the secret at the trusted request boundary; a public team ID is not a suitable salt. The salt partitions reuse, not access permissions, and does not encrypt KV. vLLM cache-sharing guidance.

Experiment B: fewer weight bytes

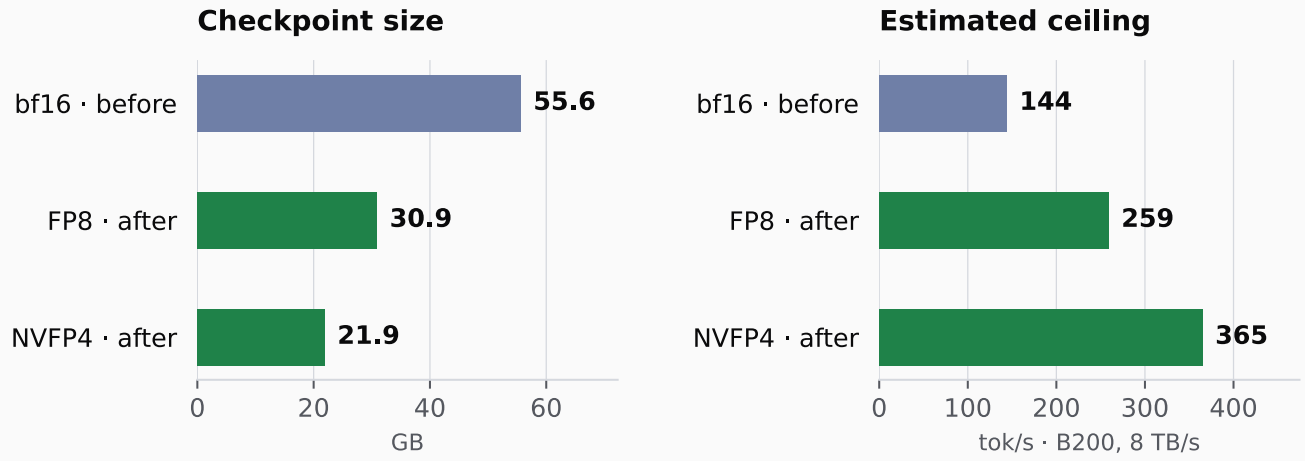
Use the same test workload and save a new result directory for each checkpoint. Online FP8 needs a supported execution path; native NVFP4 W4A4 needs Blackwell. Do not apply both changes together.

```bash
# Before: bf16 baseline, with the same context limit
vllm serve Qwen/Qwen3.8-27B --dtype bfloat16 --max-model-len 32768

# Hopper: quantize a bf16 checkpoint to FP8 at load
vllm serve Qwen/Qwen3.8-27B --max-model-len 32768 \
  --quantization fp8_per_tensor

# Blackwell: use the checkpoint's own NVFP4 quantization config
vllm serve nvidia/Qwen3.8-27B-NVFP4 --max-model-len 32768

# Benchmark the NVFP4 server; use the bf16 model ID for online FP8
vllm bench serve --backend vllm --model nvidia/Qwen3.8-27B-NVFP4 \
  --dataset-name random --random-input-len 1024 --random-output-len 512 \
  --num-prompts 80 --num-warmups 2 --max-concurrency 1 --seed 1234 \
  --save-result --result-dir results/weights-nvfp4-run1
```



Calculated, not benchmarked: Qwen3.8-27B checkpoint variants occupy 55.6 / 30.9 / 21.9 GB. Dividing B200's 8,000 GB/s by those sizes gives approximately 144 / 259 / 365 tok/s. These are checkpoint-based short-context estimates, not online-FP8 measurements. Actual bytes read, kernels and KV traffic change the result. Checkpoint sources: bf16, FP8, NVFP4.

Record `Model loading took X GiB`, actual one-user TPOT, the kernel backend and task quality. A smaller checkpoint alone does not prove a serving gain. Re-run section 5 for each precision.

Experiment C: FP8 KV at several context lengths

Run the two servers separately with prefix caching off, so repeated benchmark prompts do not turn the comparison into a prefix-cache test. Keep the weight precision fixed.

```bash
# Before: model-dtype KV (bf16 here)
vllm serve Qwen/Qwen3.8-27B --dtype bfloat16 --max-model-len 32768 \
  --no-enable-prefix-caching --kv-cache-dtype auto

# After: attention KV in FP8, hybrid fixed state remains wide
vllm serve Qwen/Qwen3.8-27B --dtype bfloat16 --max-model-len 32768 \
  --no-enable-prefix-caching --kv-cache-dtype fp8
```

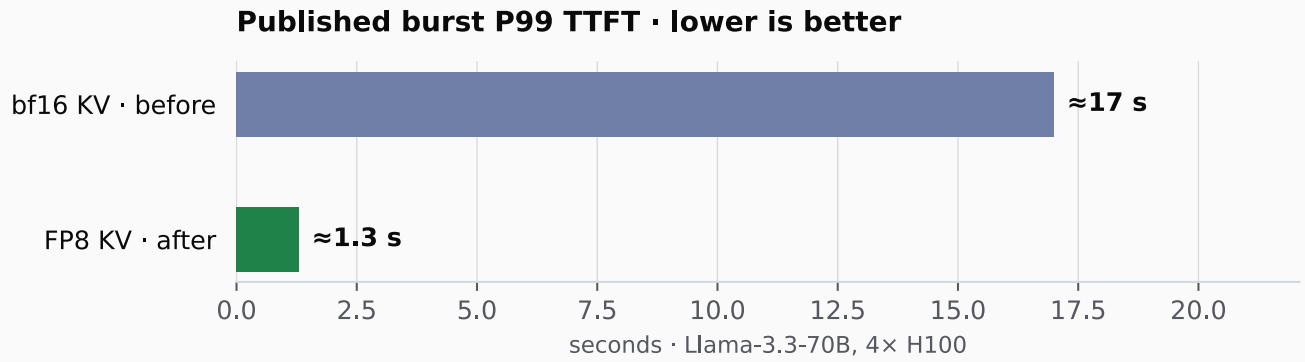
```bash
# Repeat for the FP8 server with results/kv-fp8-* directories
for BENCH_CONTEXT in 1024 8192 24576; do
  vllm bench serve --backend vllm --model Qwen/Qwen3.8-27B \
    --dataset-name random --random-input-len "$BENCH_CONTEXT" \
    --random-output-len 512 --num-prompts 80 --num-warmups 2 \
    --max-concurrency 8 --seed 1234 \
    --percentile-metrics ttft,tpot --metric-percentiles 50,95 \
    --save-result --result-dir "results/kv-bf16-$BENCH_CONTEXT"
done
```

Record startup KV token capacity, Mean TPOT, P95 TTFT and preemptions. Context plus output must fit `--max-model-len`. Lower TPOT at long context is a decode win; lower TTFT under a burst can instead be a capacity/queueing win. With sliding-window models, test `--kv-cache-dtype-skip-layers sliding_window` separately. If you see a scale-factor warning, check quality before keeping the change.

At short context, weights may dominate the read so strongly that smaller KV barely changes decode speed. With longer context or more concurrent requests, the KV read and cache capacity matter more. Recurrent state can remain wide even when attention KV is quantized, so total memory savings need not follow the bit-count ratio.

Compare formats at the same context cap, scheduling budget, cache state and weight precision. Record both cache bytes and startup token capacity. Check how many tokens were actually uncached; a high hit percentage can still leave substantial prefill work on a long input.

Check what your benchmark actually measured. A long-context sweep at low concurrency may never create cache-capacity pressure.



Published measurement, different model: Llama-3.3-70B on 4× H100, vLLM 0.20.2, 300 measured prompts, five warm-ups, 1,024 input / 512 output, burst arrivals. P99 TTFT fell from approximately 17 s with bf16 KV to 1.3 s with FP8 KV. This illustrates reduced queueing under memory pressure; it is not a Qwen3.8 result or a universal speedup. vLLM study, Figure 14.

Experiment D: speculation on saved task prompts

Start with a compatible speculator, or try n-gram lookup without training. The baseline is the same target model without `--speculative-config`. Restart between configurations and match the warm-up/cache policy.

```bash
# Trained head for this target model
vllm serve Qwen/Qwen3.8-27B --max-model-len 32768 \
  --speculative-config '{"method":"dspark","model":"RedHatAI/Qwen3.8-27B-speculator.dspark",
    "num_speculative_tokens":3}'

# Alternative trained speculator: replace the placeholder with a compatible repo
vllm serve Qwen/Qwen3.8-27B --max-model-len 32768 \
  --speculative-config '{"method":"eagle3","model":"<speculator repo>","num_speculative_tokens":2}'

# Alternative: prompt lookup, without a draft checkpoint
vllm serve Qwen/Qwen3.8-27B --max-model-len 32768 \
  --speculative-config '{"method":"ngram","num_speculative_tokens":3,"prompt_lookup_min":2,
    "prompt_lookup_max":4}'
```

Use your own `traces.jsonl` and the replay client above, with `MODEL=Qwen/Qwen3.8-27B`, `--max-concurrency 1` and a fresh result directory for each configuration and repeat. Then repeat at production concurrency. Read Mean acceptance length and TPOT together: more accepted tokens can still lose to draft cost. Random strings are a poor n-gram workload. A prompt replay measures serving speed; it does not execute tools or establish agent success. Run the real task checks in section 5 as well.

Tune speculation and prefill budgets separately

Try a few draft widths on the same traffic. A wider draft can accept more tokens per pass while spending more time proposing or verifying them. Compare total output tok/s, per-request TPOT and first-token wait at the concurrency you intend to serve. Choose by latency and throughput, not acceptance alone.

Then hold speculation fixed and vary `--max-num-batched-tokens`, for example 1024, 4096 and 8192. Smaller prefill chunks can reduce interruptions to decode but may prolong prompt processing. Keep context length, sequence cap and cache policy fixed, and compare both TTFT and TPOT. vLLM server CLI.

Finally, test the chosen configuration at controlled request rates. Use a separate result directory for each rate and repeat. Check the achieved rate as well as the requested rate: the client concurrency cap can limit arrivals.

Experiment E: reuse a prefix beyond GPU memory

CPU or disk storage can avoid recomputing a prefix after GPU eviction. Reload must be faster than recomputation to help. On unified-memory systems, a retained CPU cache consumes the same physical memory pool as the GPU; disk reuse can be more useful, with a bounded staging buffer for transfers.

For **native CPU offload with router-visible events**, use the following flags. The router also needs to consume the event stream; publishing alone does not configure routing.

```bash
vllm serve Qwen/Qwen3.8-27B --max-model-len 32768 \
  --kv-offloading-size 64 \
  --kv-transfer-config \
  '{"kv_connector_extra_config":{"self_describing_kv_events":true}}' \
  --kv-events-config '{"enable_kv_cache_events":true,"publisher":"zmq"}'
```

For **LMCache disk reuse**, use the separate setup below. For Qwen hybrid models with LMCache 0.5.6 and vLLM 0.32.0, use `--mamba-cache-mode align`. Use LMCache's hybrid-model recipe and disk and buffer-only configuration for the installed release. With a standalone multi-process LMCache server, configure disk capacity there; vLLM's `--kv-offloading-size` controls its native offload path. Match the LMCache chunk to the resolved hybrid block size in the startup log. Keep the model, KV format, backend and chunk layout fixed across a restart; use a new disk directory when that layout changes.

1. **Cold and GPU-warm:** send a fresh shared prefix containing an answer you can check, then repeat it immediately. Save the exact requests, answers and first-token waits.
2. **Store:** wait for pending disk stores to finish. Check completed objects and pending stores through LMCache's configured metrics endpoint.
3. **Restart:** stop both vLLM and LMCache so neither GPU KV nor LMCache RAM can serve the next request. Preserve the disk directory and restart the same configuration.
4. **Reload:** replay the exact requests. Measure filesystem reads as well as cache hits. If you need to distinguish disk from the OS page cache, use a supported direct-I/O path; reads still pass through host memory.
5. **Check correctness and memory:** verify every answer and confirm staging memory is released. If testing disk-only retention, verify that the retained CPU tier is empty. Compare reload TTFT with cold recomputation, including any store cost the workload cannot amortize.

Bound transfer buffers separately from retained cache capacity. Test failure and eviction behavior before relying on the tier in production.

05 Gate quality

```bash
uv pip install "lm-eval[api]==0.4.13"
EVAL_ARGS="model=Qwen/Qwen3.8-27B"
EVAL_ARGS+=",base_url=http://localhost:8000/v1/completions"
EVAL_ARGS+=",num_concurrent=16,max_retries=3,tokenized_requests=False"
lm-eval run --model local-completions --model_args "$EVAL_ARGS" \
  --tasks gsm8k --num_fewshot 5 --seed 1234 --log_samples --output_path results/baseline-run1
```

For a complete chat-model comparison, keep evaluation settings fixed. Set `model=` to the served ID from `curl -s localhost:8000/v1/models`; a quantized checkpoint can have a different ID. Use a new output directory and the planned matched seed for each run. This is your frozen zero-shot test, not a reproduction of the model-card scores.

```bash
EVAL_ARGS="model=Qwen/Qwen3.8-27B"
EVAL_ARGS+=",base_url=http://localhost:8000/v1/chat/completions"
EVAL_ARGS+=",num_concurrent=8,max_retries=3,tokenized_requests=False"
lm-eval run --model local-chat-completions --apply_chat_template \
  --model_args "$EVAL_ARGS" \
  --tasks mmlu_pro --num_fewshot 0 --seed 1234 --log_samples \
  --output_path results/quality-baseline-run1
```

Before testing, set the allowed quality loss δ in percentage points for each required metric. Repeat both baseline and candidate with matched tasks, arrival pattern, decoding settings, warm-up and cache state. For cold-cache tests, restart and clear persistent tiers too; for warm-cache tests, replay identical warm-up and exclude it from measurement. Two runs each are a first repeatability check, not a noise bound.

Use saved per-case outcomes for a paired 95% bootstrap interval of `candidate − baseline`, resampling independent tasks while keeping paired outcomes and repeated attempts together. For sampling, repeat matched seeds and check seed sensitivity. A single-score standard error is not an interval for the difference. **Pass:** lower endpoint at least $-\delta$. **Fail:** upper endpoint below $-\delta$, so revert. **Inconclusive:** interval crosses $-\delta$; keep the baseline and gather evidence. For example, with a one-point loss budget, an interval of −0.6 to +0.2 passes; −1.3 to −0.2 is inconclusive; −1.5 to −1.1 fails.

MMLU-Pro tests knowledge. Add a slice of real tasks with a separately chosen budget, because an exam alone does not test an agent. Never infer equivalence from an insignificant score change.

Test what your users need

Use tasks that cover tool calls, output schemas, long histories and failure handling. Keep the checker fixed before comparing models. Specify tuple-versus-list output, required error types and numeric tolerances explicitly; otherwise the test can score formatting accidents instead of useful behavior.

A dozen successful function completions is a smoke check, not a quality gate for a coding agent. Run independent tasks, retain per-task failures and include the longest contexts you expect to serve. Published model-card scores help select candidates, but do not validate your KV precision, speculative settings or application.

06 Dashboards and alerts

```promql
# TTFT p95
histogram_quantile(0.95, sum by (le) (rate(vllm:time_to_first_token_seconds_bucket[5m])))
# stream-output gap (ITL) p95; outputs may bundle speculative tokens
histogram_quantile(0.95, sum by (le) (rate(vllm:inter_token_latency_seconds_bucket[5m])))
# per-request TPOT p95 (use this one with speculative decoding)
histogram_quantile(0.95, sum by (le) (rate(vllm:request_time_per_output_token_seconds_bucket[5m])))
# KV cache usage (1 = 100%) and queue depth
max(vllm:kv_cache_usage_perc)
sum(vllm:num_requests_waiting)
# prefix token hit rate (cached queried tokens / queried tokens, not requests)
sum(rate(vllm:prefix_cache_hits_total[5m])) / sum(rate(vllm:prefix_cache_queries_total[5m]))
# speculative decoding: mean acceptance length
1 + sum(rate(vllm:spec_decode_num_accepted_tokens_total[5m])) /
    sum(rate(vllm:spec_decode_num_drafts_total[5m]))
# generation throughput, tok/s
sum(rate(vllm:generation_tokens_total[1m]))
```

Alert on KV usage, queue depth, P95 TTFT and P95 per-request TPOT. Keep ITL for stream smoothness; speculative output can bundle several tokens, so its histogram samples differ from TPOT's per-request samples. Prefix hit targets count tokens, not requests. Compute MBU from a one-user run, and cost from total throughput against your rent.

For hardware-level observation, dcgm-exporter exposes `DCGM_FI_PROF_DRAM_ACTIVE`: the ratio of cycles the memory interface is sending or receiving data. It measures busy memory cycles per GPU; MBU estimates bytes transferred relative to peak bandwidth. `DCGM_FI_DEV_GPU_UTIL` does not tell a busy GPU from an overloaded one.

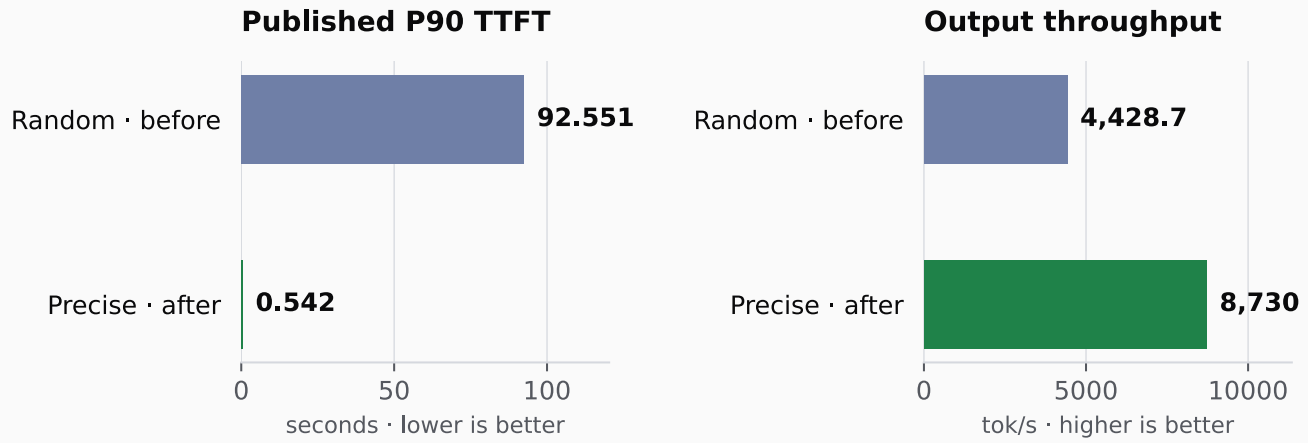
```promql
avg by (Hostname, gpu) (DCGM_FI_PROF_DRAM_ACTIVE)
```

07 The fleet: llm-d

The examples here use llm-d v0.10.0. Use the deployment guide matching your installed release.

Start with the `optimized-baseline` guide (branch `release-0.10`): Gateway API Inference Extension CRDs, then the router chart `oci://ghcr.io/llm-d/charts/llm-d-router-standalone` v0.11.0, then the vLLM pods. Full setup: optimized-baseline guide.

The Endpoint Picker reads each pod's queue and KV usage every `--refresh-metrics-interval` (default 50 ms); these metrics show how full a cache is. Precise prefix-cache routing also tracks which prefixes it holds. With one pod, the router has no placement choice. Without repeated prefixes, prefix affinity adds little; load-aware routing may still help. The same prefix-aware pattern exists in NVIDIA Dynamo (`--router-mode kv`) and the vLLM Production Stack (prefix-aware routing).



Published measurement: eight pods × two H100, 150 tenants with 6,000-token shared prefixes. Random versus precise prefix routing: P90 TTFT **92.551 → 0.542 s**; output throughput **4,428.7 → 8,730 tok/s**. The workload stresses cache locality; this is an example of the effect, not a forecast. llm-d benchmark.

Routing, KV-cache sharing and disaggregated P/D move different things. Prefix-aware routing sends the request to a replica already holding its prefix. Sharing transfers computed KV.

Direct GPU transfer: GPU → NIC → network → NIC → GPU. NIXL initiates the transfer; UCX is its default backend for on-premises RoCE and InfiniBand. With compatible hardware and registered GPU memory, GPUDirect RDMA moves the payload without staging in system RAM. llm-d's networking guide documents this path for prefill/decode. The connector coordinates the blocks; transport support alone does not locate arbitrary cached prefixes.

CPU-offloaded prefix sharing is a separate implementation. The experimental llm-d P2P path pulls matching blocks from a peer's CPU tier over NIXL, then loads the recipient's GPU. That CPU-to-CPU path is deliberate, not a GPUDirect fallback. Shared filesystem storage is another option. Match prefix tokens, model and cache layout, respect the sharing boundary, and measure loading against recomputation.

Disaggregated prefill/decode assigns prompt processing to prefill pods, then transfers the newly computed KV to decode pods for that request. It adds a pool and transfer path; use it for long prompts and strict first-token targets after tuning chunked prefill, rather than assuming every workload benefits.

Before and after a routing change, replay the same prompts and arrival trace with the same sharing and isolation policy. Record per-pod hit rate, TTFT, TPOT and queue depth. A fleet-wide average can conceal one overloaded warm pod. Include router overhead in the measured endpoint; do not compare a direct-to-pod baseline against a routed candidate without recording that difference.

Scale on request demand: GPU utilization alone cannot distinguish busy from overloaded. The release-0.10 KEDA/EPP guide pairs queue depth with running requests as separate triggers. Its router overlay enables flow control, which holds excess requests before dispatch. Without it, the queue series is absent.

```promql
# Separate KEDA triggers; selectors assume the guide's EPP service
sum(llm_d_epp_flow_control_queue_size{
  namespace="llm-d-optimized-baseline", service="optimized-baseline-epp"
})
sum(llm_d_epp_request_running{
  namespace="llm-d-optimized-baseline", service="optimized-baseline-epp"
})
```

Inspect the live Prometheus series and scope each query to the pool being scaled. Queue depth shows waiting demand; running requests preserve capacity while work is active. Load-test both thresholds against TTFT and TPOT targets. Set warm capacity and stabilization with model startup time in mind: a new pod may take minutes to serve. Use the guide for the installed release; its metrics and defaults can change.

08 Eval recipe and change ticket

1. Freeze: prompts, traces, checks, serving setup and latency targets. Set each quality-loss budget `δ` before changing one thing.

2. Repeat both: baseline and candidate, initially at least twice each; match traffic, warm-up and cache state, including persistent tiers. Record tok/s, short-context non-speculative MBU, P95 TTFT and per-request TPOT, exam and task scores, and cost per million output tokens: `GPU $/hour ÷ (output tok/s × 3,600) × 1,000,000`.

3. Compare: paired score-difference intervals, repeating matched seeds for sampled evaluations. Gate the exam and real tasks separately using section 5. Two-run gaps and single-score standard errors are diagnostics, not pass thresholds.

4. Verdict: keep only a repeatable throughput, latency or cost gain within the same latency targets after all quality gates pass. No repeatable gain or a failed gate: revert. Inconclusive: keep the baseline and gather evidence.

09 Research papers

Further reading on the mechanisms behind these experiments. Links open the primary publication pages.

1. **LLM Inference Unveiled: Survey and Roofline Model Insights**. arXiv:2402.16363.
2. **Efficient Memory Management for Large Language Model Serving with PagedAttention**. arXiv:2309.06180.
3. **Opportunistic Expert Activation: Batch-Aware Expert Routing for Faster Decode Without Retraining**. arXiv:2511.02237.
4. **Lynx: Enabling Efficient MoE Inference through Dynamic Batch-Aware Expert Selection**. arXiv:2411.08982.
5. **AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration**. arXiv:2306.00978.
6. **Four Over Six: More Accurate NVFP4 Quantization with Adaptive Block Scaling**. arXiv:2512.02010.
7. **QServe: W4A8KV4 Quantization and System Co-design for Efficient LLM Serving**. arXiv:2405.04532.
8. **A White Paper on Neural Network Quantization**. arXiv:2106.08295.
9. **KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache**. arXiv:2402.02750.
10. **KVQuant: Towards 10 Million Context Length LLM Inference with KV Cache Quantization**. arXiv:2401.18079.
11. **QAQ: Quality Adaptive Quantization for LLM KV Cache**. arXiv:2403.04643.
12. **KVTuner: Sensitivity-Aware Layer-Wise Mixed-Precision KV Cache Quantization for Efficient and Nearly Lossless LLM Inference**. arXiv:2502.04420.
13. **Fast Inference from Transformers via Speculative Decoding**. arXiv:2211.17192.
14. **EAGLE-3: Scaling up Inference Acceleration of Large Language Models via Training-Time Test**. arXiv:2503.01840.
15. **Splitwise: Efficient generative LLM inference using phase splitting**. arXiv:2311.18677.
16. **DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving**. arXiv:2401.09670.
17. **DeepSeek-V3 Technical Report**. arXiv:2412.19437.
18. **DeepSeek-V3.2: Pushing the Frontier of Open Large Language Models**. arXiv:2512.02556.
19. **DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence**. arXiv:2606.19348.
20. **DeepSeek-V4.1-Flash: Pushing the Limits of KV Cache Compression**. Technical report (PDF).